

Final Project Part 2

Part 1: Integrating Data from an API

The support-ticket API is provided as a local mock server. Start it with `python3 mock_api.py`, then read from `http://127.0.0.1:8000/v1/tickets?page=1`.

```
Terminal Local x Local (2) x + v
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

(.venv) PS C:\Users\aylee\PyCharmMiscProject> python mock_api.py
C:\Users\aylee\PyCharmMiscProject\.venv\Scripts\python.exe: can't open file 'C:\Users\aylee\PyCharmMiscProject\mock_api.py': [Errno 2] No such file or directory

(.venv) PS C:\Users\aylee\PyCharmMiscProject> Get-ChildItem -Path C:\Users\aylee -Filter mock_api.py -Recurse -ErrorAction SilentlyContinue

Directory: C:\Users\aylee\Downloads

Mode                LastWriteTime         Length Name
----                -
-a-----         2026-08-08  12:32 PM             2256 mock_api.py

(.venv) PS C:\Users\aylee\PyCharmMiscProject>
(.venv) PS C:\Users\aylee\PyCharmMiscProject> cd C:\Users\aylee\Downloads
(.venv) PS C:\Users\aylee\Downloads> python mock_api.py
mock API on http://127.0.0.1:8000/v1/tickets?page=1 (ctrl-c to stop)
API 127.0.0.1 "GET /v1/tickets?page=1 HTTP/1.1" 404 -

+ FullyQualifiedPathName : PathNotFound,Microsoft.PowerShell.Commands.GetChildItemCommand

(.venv) PS C:\Users\aylee\PyCharmMiscProject> Get-ChildItem C:\Users\aylee\Downloads -Filter *.zip

Directory: C:\Users\aylee\Downloads

Mode                LastWriteTime         Length Name
----                -
-a-----         2026-08-08  12:32 PM             3970 api_pages.zip
-a-----         2026-03-06   4:27 PM             31835 aRCpkrL0Tuyh9x2mHWg_5g_4d4dc230247e4abcbae887602bb3f4e1_AdventureWorksSales (1).zip
-a-----         2026-03-09   9:45 AM             31835 aRCpkrL0Tuyh9x2mHWg_5g_4d4dc230247e4abcbae887602bb3f4e1_AdventureWorksSales (2).zip
-a-----         2026-03-06   4:20 PM             31835 aRCpkrL0Tuyh9x2mHWg_5g_4d4dc230247e4abcbae887602bb3f4e1_AdventureWorksSales.zip
-a-----         2026-04-01   3:31 PM              631 Assignment 1.Task 4.zip
-a-----         2026-04-17  11:18 AM             3153 Assignment 3.zip
-a-----         2026-04-28  12:57 PM             13205 Assignment 4.zip
-a-----         2026-03-30   6:23 PM              4413 Assignment 1.zip
-a-----         2026-03-30   6:24 PM             3932 Assignment 2.zip
-a-----         2026-04-01  10:27 AM             1299 Assignment 3 (1).zip
-a-----         2026-03-30   6:25 PM             1299 Assignment 3.zip
-a-----         2026-03-30   6:25 PM             3063 Assignment 4.zip
-a-----         2026-03-30   6:25 PM             3552 Assignment 5.zip
-a-----         2026-05-01   9:38 AM             3850004 attachments (1).zip
-a-----         2026-05-05   9:39 AM             887866 attachments (2).zip
-a-----         2026-04-17   3:08 PM             1889171 attachments.zip
-a-----         2026-04-17   4:15 PM             137161998 attachments (1).zip
```

```
-a---- 2026-03-31 3:19 PM 3779 Lesson 7 - Data Manipulation in Python Using Pandas.zip
-a---- 2026-04-01 3:50 PM 4154 Lesson 8 - Data Manipulation in Python Using Pandas II (1).zip
-a---- 2026-04-01 2:44 PM 4154 Lesson 8 - Data Manipulation in Python Using Pandas II.zip
-a---- 2026-07-09 1:13 PM 61965279 mongosh-2.9.2-win32-x64.zip
-a---- 2026-04-04 8:42 AM 2506025 Student Resources - ACSW-20260404.zip
-a---- 2026-04-04 8:43 AM 283104 Student Resources DTABIAAAI-20260404.zip
-a---- 2026-04-08 9:24 AM 308301 Student Resources DTABIAAAI-20260408.zip
-a---- 2026-04-10 10:25 AM 321017 Student Resources DTABIAAAI-20260410.zip
-a---- 2026-04-04 8:38 AM 2046284 Student Resources-20260404.zip
-a---- 2026-04-07 12:35 PM 1851129 Student Resources-20260407 (1).zip
-a---- 2026-04-07 12:15 PM 16922977 Student Resources-20260407.zip
-a---- 2026-04-08 9:42 AM 3561098 Student Resources-20260408.zip
-a---- 2026-04-09 9:40 AM 3561098 Student Resources-20260409.zip

(.venv) PS C:\Users\aylee\PyCharmMiscProject> Get-ChildItem C:\Users\aylee\Downloads\api_pages -Recurse
(.venv) PS C:\Users\aylee\PyCharmMiscProject> Expand-Archive -Path C:\Users\aylee\Downloads\api_pages.zip -DestinationPath C:\Users\aylee\Downloads -Force
(.venv) PS C:\Users\aylee\PyCharmMiscProject> Get-ChildItem C:\Users\aylee\Downloads\api_pages

Directory: C:\Users\aylee\Downloads\api_pages

Mode                LastWriteTime         Length Name
----                -
-a----            2026-08-07 11:11 AM           7733 page1.json
-a----            2026-08-07 11:11 AM           7779 page2.json
-a----            2026-08-07 11:11 AM           6807 page3.json
-a----            2026-08-07 11:11 AM           104 rate_limited.json

(.venv) PS C:\Users\aylee\PyCharmMiscProject>

Terminal  Local x Local (2) x Local (3) x Local (4) x Local (5) x Local (6) x + v

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

(.venv) PS C:\Users\aylee\PyCharmMiscProject> cd C:\Users\aylee\Downloads
(.venv) PS C:\Users\aylee\Downloads> cd C:\Users\aylee\Downloads

(.venv) PS C:\Users\aylee\Downloads> python mock_api.py
mock API on http://127.0.0.1:8000/v1/tickets?page=1 (ctrl-c to stop)
Traceback (most recent call last):
  File "C:\Users\aylee\Downloads\mock_api.py", line 57, in <module>
    HTTPServer(('127.0.0.1', 8000), Handler).serve_forever()
  File "C:\Users\aylee\AppData\Local\Python\pythoncore-3.14-64\Lib\socketserver.py", line 235, in serve_forever
    ready = selector.select(poll_interval)
  File "C:\Users\aylee\AppData\Local\Python\pythoncore-3.14-64\Lib\selectors.py", line 314, in select
    r, w, _ = self._select(self._readers, self._writers, [], timeout)
  File "C:\Users\aylee\AppData\Local\Python\pythoncore-3.14-64\Lib\selectors.py", line 305, in _select
    r, w, x = select.select(r, w, w, timeout)
KeyboardInterrupt

C:\Users\aylee\PyCharmMiscProject\.venv\Scripts\python.exe "C:\Users\aylee\PyCharmMi
Successfully fetched the page!
{'meta': {'page': 1, 'per_page': 25, 'total_pages': 3, 'total_records': 72, 'next':

Process finished with exit code 0
```

1. **Fetch every ticket.** The API returns 25 per page and tells you in `meta.next` where the next page is. Follow that field rather than hard-coding the number of pages; a feed that grows next week should not need a code change.

```

3  # /*Part 1: Integrating Data from an API*/
4  import requests
5  import json
6
7  response = requests.get("http://127.0.0.1:8000/v1/tickets?page=1")
8  data = response.json()
9  print(json.dumps(data, indent=2))
10
11 if response.status_code == 200:
12     print("Successfully fetched the page!")
13     data = response.json()
14     print(data)
15 else:
16     print(f"Failed to retrieve the page: {response.status_code}")
17 print('--'*30)

```

```

}
Successfully fetched the page!
{'meta': {'page': 1, 'per_page': 25, 'total_pages': 3, 'total_records': 72, 'next': '/v1/tic
-----

```

```

    "satisfaction_score": null
  },
  {
    "ticket_id": "TK-4023",
    "opened_at": "2025-04-09",
    "customer": {
      "email": "clara.duarte@corvid.io"
    },
    "product_sku": "PC-3022",
    "issue_type": "refund_request",
    "status": "resolved",
    "resolution_days": 19,
    "satisfaction_score": 4
  },
  {
    "ticket_id": "TK-4024",
    "opened_at": "2024-11-06",
    "customer": {
      "email": "teo.petrov@mailhaven.com"
    },
    "product_sku": "HK-1090",
    "issue_type": "delivery",
    "status": "resolved",
    "resolution_days": 14,
    "satisfaction_score": 4
  }
]
}

```

2. **Handle the rate limit.** One request will come back with HTTP 429 and a **Retry-After** header instead of data. Wait and retry rather than crashing or skipping the page. A run that quietly loses 25 tickets is worse than one that fails loudly.

```

35 # 2.Handle rate limiting: wait the requested time, then retry the SAME page
36 if response.status_code == 429:
37     retry_after = int(response.headers.get("Retry-After", 1))
38     print(f"Rate limited on {next_path}. Waiting {retry_after}s before retry...")
39     time.sleep(retry_after)
40     continue
41
42 if response.status_code == 404:
43     print(f"Page not found: {next_path}")
44     break
45
46 response.raise_for_status()
47 payload = response.json()
48
49 # Find the list of tickets regardless of what the key is actually called
50 ticket_list = None
51 for key, value in payload.items():
52     if isinstance(value, list):
53         ticket_list = value
54         ticket_key = key
55         break
56
57 if ticket_list is None:
58     print("Could not find a ticket list in this page. Full payload:")
59     print(payload)
60     break
61
62 all_tickets.extend(ticket_list)
63 print(f"Fetchd {next_path} - {len(ticket_list)} tickets under key '{ticket_key}' "
64       f"(running total: {len(all_tickets)})")
65
66 next_path = payload.get("meta", {}).get("next")
67
68 print(f"\nDone. Total tickets fetched: {len(all_tickets)}")
69
70 print(f"\nDone. Total tickets fetched: {len(all_tickets)}")
71 if all_tickets:
72     print("\nTicket (first record):")
73     print(all_tickets[0])
74     print('\n--'*30)

```

```

-----
Fetchd /v1/tickets?page=1 - 25 tickets under key 'data' (running total: 25)
Fetchd /v1/tickets?page=2 - 25 tickets under key 'data' (running total: 50)
Fetchd /v1/tickets?page=3 - 22 tickets under key 'data' (running total: 72)

Done. Total tickets fetched: 72

Ticket (first record):
{'ticket_id': 'TK-4000', 'opened_at': '2024-01-10', 'customer': {'email': 'sanne.devries@bric
-----

```

3. **Handle the optional fields.** Open tickets have no `resolution_days`, and only resolved tickets carry a `satisfaction_score`. Decide what goes in the database for a value that does not exist yet, and say why that is different from a value of zero.

```
Terminal Local (2) x Local (3) x Local (4) x Local (5) x Local (6) x Unnamed x + v
(.venv) PS C:\Users\aylee\PyCharmMiscProject> cd "C:\Users\aylee\PyCharmMiscProject\9 Intr Data, Databases & ETL Foundations\Final Project"
(.venv) PS C:\Users\aylee\PyCharmMiscProject\9 Intr Data, Databases & ETL Foundations\Final Project> Rename-Item -Path ".env.py" -NewName ".env"
(.venv) PS C:\Users\aylee\PyCharmMiscProject\9 Intr Data, Databases & ETL Foundations\Final Project> Get-ChildItem -Force
```

Directory: C:\Users\aylee\PyCharmMiscProject\9 Intr Data, Databases & ETL Foundations\Final Project

Mode	LastWriteTime	Length	Name
-a----	2020-08-06 1:39 PM	84	.env
-a----	2020-08-06 12:46 PM	933	customer_surveys.csv
-a----	2020-08-06 4:35 PM	945	customer_surveys_cleaned.csv
-a----	2020-08-05 6:26 PM	1389	external_reviews.xml
-a----	2020-08-08 2:22 PM	3472	Final Project Part 2.py
-a----	2020-08-06 4:35 PM	359	internal_reviews_cleaned.csv
-a----	2020-08-08 2:01 PM	8043	page1_dump.json
-a----	2020-08-07 3:14 PM	2298	Part 2 Data Integration updated.py
-a----	2020-08-06 1:00 PM	1549	Part 3 Data Integration.py
-a----	2020-08-06 4:31 PM	6357	Part 4 Data Parsing.py
-a----	2020-08-07 12:05 PM	17923	web_feedback.json
-a----	2020-08-07 3:14 PM	7005	web_feedback_from_json.csv

4. (.venv) PS C:\Users\aylee\PyCharmMiscProject\9 Intr Data, Databases & ETL Foundations\Final Project> █
5. **Load into a staging table**, then into a permanent **support_ticket** table in your Project 1 database. Match each ticket to an existing customer by email and product by SKU, using the same approach you already built.

```
156 /*Final Project part 2, Part 1: Integrating Data from an API,
157 4. Load into a staging table, then into a permanent support_ticket table in your Project
158 1 database. Match each ticket to an existing customer by email and product by SKU,
159 using the same approach you already built.*/
```

```
160 • DROP TABLE IF EXISTS stg_support_ticket;
161 • CREATE TABLE stg_support_ticket (
162     ticket_id          VARCHAR(20),
163     opened_at          VARCHAR(20),
164     product_sku        VARCHAR(20),
165     issue_type          VARCHAR(50),
166     status              VARCHAR(20),
167     resolution_days     VARCHAR(20),
168     ...
```

```
188 • ALTER TABLE customers RENAME COLUMN email TO customer_email;
189 • SELECT
190     st.ticket_id,
191     st.product_sku,
192     st.customer_email,
193     c.name AS Customer_Name,
194     p.SKU AS Product_SKU,
195     p.name AS Product_Name
196 FROM stg_support_ticket st
197 LEFT JOIN customers c
198     ON LOWER(TRIM(st.customer_email)) = LOWER(TRIM(c.customer_email))
199 LEFT JOIN products p
200     ON LOWER(TRIM(st.product_sku)) = LOWER(TRIM(p.SKU));
```

6. **Compare the API to the file sources** in a short paragraph: what was harder about it, and what would break if the API were unavailable when your load ran.

```
/*5.Compare the API to the file sources in a short paragraph: what was harder about it, and what would break if the API were unavailable when your load ran.*/
/*File sources CSV,JSON and XML sources in this project has a difficult part when I tried to fetch in MySQL (JSON, XML) with LOAD LOCAL INFILE option and got the error 'disable security option' and that took me long time to discover the right path and how remove the option into the terminal and after that was quick to move forward, also I journey in the entire MYSQL to disable, and the when I understood why we need to applied this process was more reasonable, the only things it was to disable again, that was another extra time to turn OFF again.
```

Instead the API's it was complicated find the mock_api files, in the begining I tought was the regular instalation, it was super complicated to find the right way also agains time but how I had alreary experience with the other files with similar process, I imagined it was similar so I tried to find the information in the regular package in Python but I wasnt the right way and then when I search information about the terminal and search into the powarshell, that part to get the right folder and enter into mock folder and then understand to attached into my final project files was a ticky. I wasnt a regular API fetching also this technique I was to apply also in the middle the coding and actived the most hard was to fetch and find the right key words in json file,and in Python find the right way to active the mock file, the funny things the instructions mentioned after I find the folder in the terminal write the script in Python and I write the sript in the terminal and I got errors and then I understood I write my script in MyPythonCharm and run. In my consideration, both process were complicated because I am not familiarized with this process, but It was good practice and learn how can simulate the apis in Python*/

026

027 • SHOW GLOBAL VARIABLES LIKE 'local_infile';

028

Result Grid Filter Rows: Export: Wrap Cell Content:

	Variable_name	Value
▶	local_infile	ON

```
C:\Windows\system32\cmd.e: X + v
C:\Users\aylee>cmd
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\aylee>"C:\Program Files\MySQL\MySQL Server 8.0"
"C:\Program Files\MySQL\MySQL Server 8.0" is not recognized as an internal or external command,
operable program or batch file.

C:\Users\aylee>"C:\Program Files\MySQL\MySQL Server 8.0"
"C:\Program Files\MySQL\MySQL Server 8.0" is not recognized as an internal or external command,
operable program or batch file.

C:\Users\aylee>cd /d "C:\Program Files\MySQL\MySQL Server 8.0\bin"

C:\Program Files\MySQL\MySQL Server 8.0\bin>mysql --local-infile=1 -u root -p
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 31
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

```

027 • SHOW GLOBAL VARIABLES LIKE 'local_infile';
028 • SET GLOBAL local_infile = 0;
029
030 /*Part 2: Designing the Warehouse
031 Design a star schema for feedback analysis. You
032 set of dimensions. Draw it, a simple diagram is

```

Output

Action Output

#	Time	Action
947	15:44:57	SELECT COUNT(*) AS total_tickets, SUM(CASE WHEN customer_id IS NULL THEN
948	15:45:18	SELECT COUNT(*) AS total_tickets, SUM(CASE WHEN c.customerid IS NULL THEN
949	07:42:15	SHOW GLOBAL VARIABLES LIKE 'local_infile'
950	07:48:34	SET GLOBAL local_infile = 0

```

027 • SHOW GLOBAL VARIABLES LIKE 'local_infile';
028 • SET GLOBAL local_infile = 0;
029 • SHOW GLOBAL VARIABLES LIKE 'local_infile';

```

Result Grid

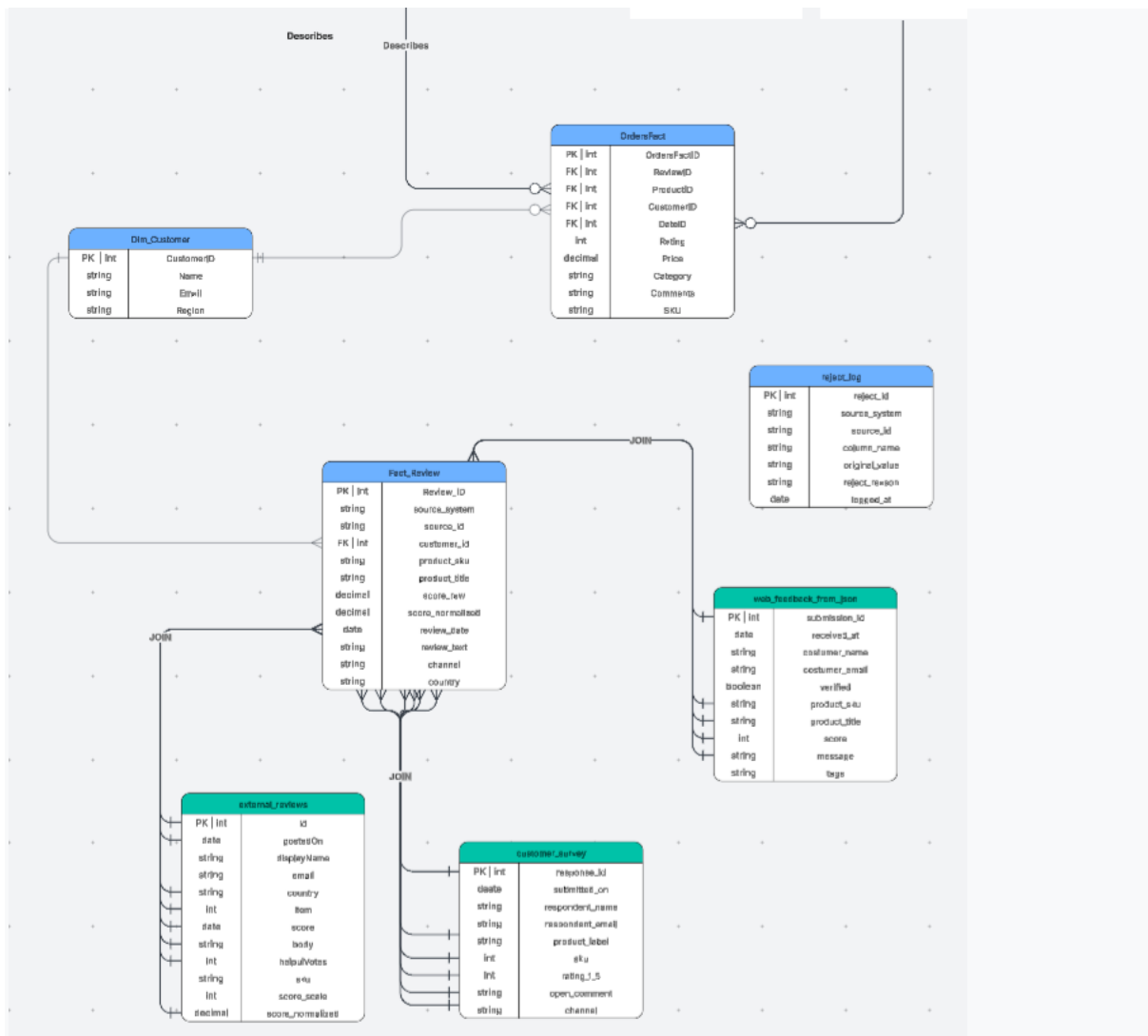
Filter Rows: Export: Wrap Cell Content:

Variable_name	Value
local_infile	OFF

Part 2: Designing the Warehouse

1. **Design a star schema** for feedback analysis. You need one fact table and a set of dimensions. Draw it, a simple diagram is fine; it does not need to be a full ER diagram.

Entire view of Data Warehouse include all fact and dimensions.



2. **State the grain** of your fact table in one sentence: what does exactly one row represent? Get this wrong, and every number the warehouse produces will be wrong in the same way.

OrdersFact represents a combination of 3 dimensions, dim_customer, dim_product and dim_date

Fact_Review represents and get information from dim_customer, and combined 3 different tables with different informations and from different sources: external_reviews_from_xml, web_feedback_from_json and customer_survey_from_csv. The Fact table contain 132 rows, we can retrieve enough information from those tables.

Fact_tickets represents all dim_product, dim_customer, and dim_date focuses it shows the ticket information.

3. **Build a date dimension** covering the range of your data, including year, quarter, month, month name, and day of the week. Generate it with SQL rather than typing it.

```
026 /*3.3. Build a date dimension covering the range of your data, including year, quarter,
027 month, month name, and day of the week. Generate it with SQL rather than typing it. */
028 • SELECT * FROM Dim_Date;
029 • SELECT COUNT(*) AS TotalDays,
030         MIN(Full_Date) AS EarliestDate,
031         MAX(Full_Date) AS LatestDate
```

Result Grid

DateID	Full_Date	Year	Quarter	Month	MonthName	DayOfWeek
20260512	2026-05-12	2026	2	5	May	Tuesday
20260513	2026-05-13	2026	2	5	May	Wednesday
20260514	2026-05-14	2026	2	5	May	Thursday
20260515	2026-05-15	2026	2	5	May	Friday
20260516	2026-05-16	2026	2	5	May	Saturday
20260517	2026-05-17	2026	2	5	May	Sunday
20260518	2026-05-18	2026	2	5	May	Monday
20260519	2026-05-19	2026	2	5	May	Tuesday
20260520	2026-05-20	2026	2	5	May	Wednesday

```
026 /*3.3. Build a date dimension covering the range of your data, including year, quarter,
027 month, month name, and day of the week. Generate it with SQL rather than typing it. */
028 • SELECT * FROM Dim_Date;
029 • SELECT COUNT(*) AS TotalDays,
030         MIN(Full_Date) AS EarliestDate,
031         MAX(Full_Date) AS LatestDate
032 FROM Dim_Date;
033
```

Result Grid

TotalDays	EarliestDate	LatestDate
60	2026-05-12	2026-07-10

- DROP TABLE IF EXISTS DIM_PRODUCT;
- CREATE TABLE DIM_PRODUCT (
 - product_key INT AUTO_INCREMENT PRIMARY KEY,
 - sku VARCHAR(20) NOT NULL,
 - name VARCHAR(100),
 - category VARCHAR(50) NOT NULL,
 - price DECIMAL(10,2) NULL,
 - is_current BOOLEAN NOT NULL DEFAULT TRUE,
 - effective_date DATE NULL,
 - expiry_date DATE NULL,
 - CONSTRAINT uq_dim_product_sku UNIQUE (sku));

```

005 /*dim customer exist and only I will matcht with actual dim product,
006 Alter dim customer*/
007 • ALTER TABLE DIM_CUSTOMER CHANGE COLUMN customer_id customer_key INT NOT NULL AUTO_INCREMENT;
008 /*Populate DIM_Customer*/
009 • SELECT * FROM DIM_CUSTOMER;
010 • INSERT INTO DIM_CUSTOMER (customer_key, email, customer_name, region)
011 SELECT
012     c.customerid,
013     c.customer_email,
014

```

	customer_key	email	customer_name	region
▶	1	maria.gonzalez@example.com	Maria Gonzalez	Manitoba
	2	james.whitfield@example.com	James Whitfield	Ontario
	3	priya.sharma@example.com	Priya Sharma	British Columbia
	4	liam.oconnor@example.com	Liam O'Connor	Alberta
	5	fatima.alsayed@example.com	Fatima Al-Sayed	Quebec
	6	noah.bergeron@example.com	Noah Bergeron	Manitoba
	7	chen.wei@example.com	Chen Wei	British Columbia
	8	sofia.rossi@example.com	Sofia Rossi	Ontario
	9	ethan.campbell@example.com	Ethan Campbell	Nova Scotia
	10	aiyana.thunderbird@example.c...	Aiyana Thunderbird	Saskatchewan

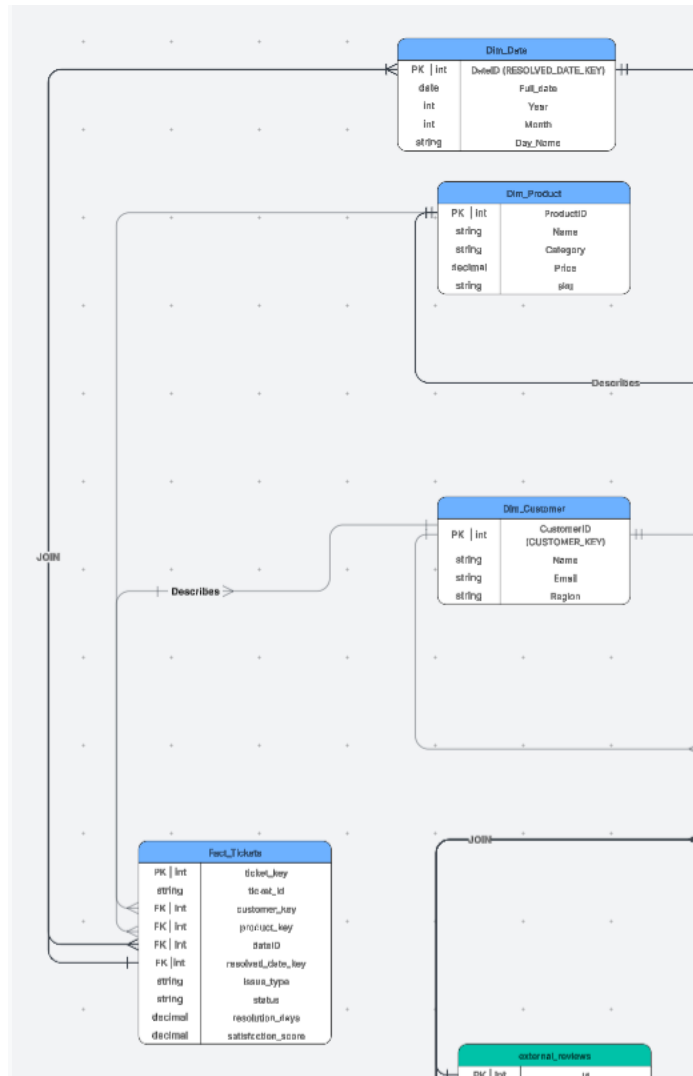
4. **Use surrogate keys** in the warehouse rather than reusing operational IDs, and explain in one paragraph why a warehouse normally does so.

```

/*Use surrogate keys in the warehouse rather than reusing operational IDs, and explain in one paragraph why a warehouse normally does so.
I applies Dim_Date.DateID already is a surrogate key in spirit (it's system-generated in the YYYYMMDD format, not borrowed from any source system).
But Reviews.ReviewID, Customers.CustomerID, and Products.ProductID are being reused directly as the join keys in OrdersFact rather than the fact table
generating its own surrogate keys for dim_customer, dim_product, etc. and mapping the operational IDs into a lookup table. */

```

5. **Add tickets as a second fact table** that shares the same product, customer, and date dimensions. Dimensions used by more than one fact table are called *conformed*; say what would go wrong if each fact table had its own copy of the product dimension instead.



```

06  /*Create a second fact table tickets*/
07  • DROP TABLE IF EXISTS Fact_Tickets;
08  • CREATE TABLE FACT_TICKETS (
09      ticket_key          INT AUTO_INCREMENT PRIMARY KEY,
10      ticket_id           VARCHAR(20) NOT NULL,
11      customer_key        INT,
12      product_key         INT,
13      DateID              INT,
14      resolved_date_key   INT,
15      issue_type          VARCHAR(50) NOT NULL,
16      status              VARCHAR(20) NOT NULL,
17      resolution_days      DECIMAL(6,1) NULL,
18      satisfaction_score   DECIMAL(3,1) NULL,
  
```

```

1024         FOREIGN KEY (DateID) REFERENCES DIM_DATE (DateID),
1025     CONSTRAINT fk_ticket_resolved_date
1026         FOREIGN KEY (resolved_date_key) REFERENCES DIM_DATE (DateID)
1027 );
1028 /* retrieve all data*/
1029 • select * from stg_support_ticket;
1030 • select * from FACT_TICKETS;
1031 • select * from dim_customer;
1032 • select * from dim_product;
1033 • select * from dim_DATE;

```

	ticket_key	ticket_id	customer_key	product_key	DateID	resolved_date_key	issue_type	status	resolution_days	satisfaction_score
▶	8	TK-4000	1	1	20240110	20240113	faulty	resolved	3.0	5.0
	9	TK-4005	6	6	20240131	20240206	faulty	resolved	6.0	3.0
	10	TK-4019	18	18	20240330	20240413	how_to	resolved	14.0	4.0
	11	TK-4010	11	11	20240501	20240513	refund_request	resolved	12.0	5.0
	12	TK-4011	12	12	20240602	20240619	faulty	resolved	17.0	4.0
	13	TK-4016	16	16	20240605		wrong_item	open		
	14	TK-4003	4	4	20240615		damaged_on_arrival	escalated	3.0	
	15	TK-4015	15	15	20240707		how_to	escalated	6.0	
	16	TK-4020	8	8	20240906	20240909	delivery	resolved	3.0	1.0
	17	TK-4002	2	2	20240010		wrong_item	escalated	15.0	

```

338 /*RESOLVE ISSUE IN TICKET UPDATE DATA*/
339 • SET SQL_SAFE_UPDATES = 0;
340 • UPDATE stg_support_ticket
341     SET resolution_days = NULL
342     WHERE resolution_days = '';
343
344 • UPDATE stg_support_ticket
345     SET satisfaction_score = NULL
346     WHERE satisfaction_score = '';
347 • SET SQL_SAFE_UPDATES = 1;

```

```

1048 • INSERT INTO FACT_TICKETS (
1049     ticket_id, customer_key, product_key, DateID, resolved_date_key,
1050     issue_type, status, resolution_days, satisfaction_score
1051 )
1052 SELECT
1053     s.ticket_id,
1054     c.customer_key,
1055     p.product_key,
1056     d.DateID,

```

	ticket_key	ticket_id	customer_key	product_key	DateID	resolved_date_key	issue_type	status	resolution_days	satisfaction_score
▶	8	TK-4000	1	1	20240110	20240113	faulty	resolved	3.0	5.0
	9	TK-4005	6	6	20240131	20240206	faulty	resolved	6.0	3.0
	10	TK-4019	18	18	20240330	20240413	how_to	resolved	14.0	4.0
	11	TK-4010	11	11	20240501	20240513	refund_request	resolved	12.0	5.0
	12	TK-4011	12	12	20240602	20240619	faulty	resolved	17.0	4.0
	13	TK-4016	16	16	20240605		wrong_item	open		
	14	TK-4003	4	4	20240615		damaged_on_arrival	escalated	3.0	
	15	TK-4015	15	15	20240707		how_to	escalated	6.0	
	16	TK-4020	8	8	20240906	20240909	delivery	resolved	3.0	1.0
	17	TK-4002	2	2	20240010		wrong_item	escalated	15.0	

FACT_TICKETS 171 x

6. **Compare the two schemas.** Take one analytical question, write it against your Project 1 normalized schema and against your star schema, and show both queries side by side. Count the joins in each.

Boths schema provide the same result in this case.

What is the average rating by product category, per quarter?

```
1077 /*6.Compare the two schemas. Take one analytical question, write it against your
1078 and show both queries side by side. Count the joins in each.*/
1079 /*Project 1 Normalized schema*/
1080 • SELECT
1081     p.category,
1082     QUARTER(r.Full_Date) AS qtr,
1083     AVG(r.rating) AS avg_rating
1084 FROM Reviews r
1085 JOIN Products p ON r.productid = p.productid
1086 GROUP BY p.category, QUARTER(r.Full_Date);
```

	category	qtr	avg_rating
▶	Electronics	2	4.7500
	Furniture	2	4.0000
	Electronics	3	4.5000
	Furniture	3	4.0000

```
1087 /*Part 2 Star schema – OrdersFact*/
1088 • SELECT
1089     o.Category,
1090     d.quarter AS qtr,
1091     AVG(o.Rating) AS avg_rating
1092 FROM OrdersFact o
1093 JOIN Dim_Date d ON o.DateID = d.DateID
1094 GROUP BY o.Category, d.quarter;
```

	Category	qtr	avg_rating
▶	Electronics	2	4.7500
	Furniture	2	4.0000
	Electronics	3	4.5000
	Furniture	3	4.0000

Part 3: Building the ETL

1. **Load in dependency order:** dimensions first, then facts, because a fact row cannot reference a dimension key that does not exist yet. Write this as a repeatable script.

```

.10  /*Part 3: Building the ETL
.11  1.Load in dependency order: dimensions first, then facts, because a fact re
.12  DELIMITER //
.13  • CREATE PROCEDURE etl_run_project2()
.14  BEGIN
.15      -- STEP 0: guarantee Unknown sentinels exist (idempotent)
.16      INSERT INTO Customers (CustomerID, Name, Region, customer_email)
.17      SELECT -1, 'Unknown Customer', 'Unknown', 'unknown@unknown.com'
.18      WHERE NOT EXISTS (SELECT 1 FROM Customers WHERE CustomerID = -1);
.19
.20      INSERT INTO Products (ProductID, name, category, price, SKU)
.21      SELECT -1, 'Unknown Product', 'Unknown', 0.00, 'UNKNOWN'
.22      WHERE NOT EXISTS (SELECT 1 FROM Products WHERE ProductID = -1);
.23
.24      INSERT IGNORE INTO dim_customer (email, customer_name, region)
.25      VALUES ('unknown@unknown.com', 'Unknown Customer', 'Unknown');
.26
.27      INSERT IGNORE INTO dim_product (sku, name, category, price, is_current,
.28      VALUES ('UNKNOWN', 'Unknown Product', 'Unknown', NULL, TRUE, CURDATE())

31      INSERT IGNORE INTO dim_customer (email, customer_name, region)
32      SELECT c.customer_email, c.name, c.region FROM Customers c;
33
34      INSERT IGNORE INTO dim_product (sku, name, category, price, effective_date
35      SELECT DISTINCT s.product_sku,
36          CONCAT('Unknown Product (', s.product_sku, ')'),
37          CASE
38              WHEN s.product_sku LIKE 'PC-%' THEN 'Personal Care'
39              WHEN s.product_sku LIKE 'OF-%' THEN 'Office'
40              WHEN s.product_sku LIKE 'EL-%' THEN 'Electronics'
41              WHEN s.product_sku LIKE 'OG-%' THEN 'Outdoor Gear'
42              WHEN s.product_sku LIKE 'HK-%' THEN 'Home & Kitchen'
43              ELSE 'Unknown'
44          END,
45          NULL, CURDATE()
46      FROM stg_support_ticket s;
47
48      -- STEP 2: facts (dimensions above are now guaranteed populated)
49      INSERT INTO OrdersFact
50          (ReviewID, CustomerID, ProductID, DateID, Rating, Price, Category, Con
51      SELECT
52          r.ReviewID,

```

```

158         COALESCE(p.Category, 'Unknown'),
159         r.Comments,
160         COALESCE(p.SKU, 'UNKNOWN')
161 FROM Reviews r
162 LEFT JOIN Products p ON r.ProductID = p.ProductID
163 LEFT JOIN Customers c ON r.CustomerID = c.CustomerID
164 LEFT JOIN Dim_Date d ON d.Full_date = DATE(r.Full_Date)
165 ON DUPLICATE KEY UPDATE
166     CustomerID = VALUES(CustomerID), ProductID = VALUES(ProductID),
167     DateID = VALUES(DateID), Rating = VALUES(Rating), Price = VALUES(Pr
168     Category = VALUES(Category), Comments = VALUES(Comments), SKU = VAL
169
170 INSERT INTO FACT_TICKETS (
171     ticket_id, customer_key, product_key, DateID, resolved_date_key,
172     issue_type, status, resolution_days, satisfaction_score
173 )
174 SELECT
175     s.ticket_id,
176     COALESCE(c.customer_key, (SELECT customer_key FROM dim_customer WHE
177     COALESCE(p.product_key, (SELECT product_key FROM dim_product WHE
178     d.DateID,
179     d_resolved.DateID,

```



```

173 )
174 SELECT
175     s.ticket_id,
176     COALESCE(c.customer_key, (SELECT customer_key FROM dim_customer WHERE e
177     COALESCE(p.product_key, (SELECT product_key FROM dim_product WHERE s
178     d.DateID,
179     d_resolved.DateID,
180     s.issue_type, s.status, s.resolution_days, s.satisfaction_score
181 FROM stg_support_ticket s
182 LEFT JOIN DIM_CUSTOMER c ON LOWER(TRIM(c.email)) = LOWER(TRIM(s.customer_en
183 LEFT JOIN DIM_PRODUCT p ON TRIM(p.sku) = TRIM(s.product_sku)
184 JOIN DIM_DATE d ON d.full_date = DATE(s.opened_at)
185 LEFT JOIN DIM_DATE d_resolved
186     ON s.status = 'resolved'
187     AND d_resolved.full_date = DATE(s.opened_at) + INTERVAL s.resolution_day
188 ON DUPLICATE KEY UPDATE
189     customer_key = VALUES(customer_key), product_key = VALUES(product_key),
190     DateID = VALUES(DateID), resolved_date_key = VALUES(resolved_date_key),
191     issue_type = VALUES(issue_type), status = VALUES(status),
192     resolution_days = VALUES(resolution_days), satisfaction_score = VALUES(
193 END //
194 DELIMITER ;

```

Output					
Action Output					
#	Time	Action	Message	Duration / Fetch	
✓ 597	09:57:16	SELECT * FROM FACT_TICKETS LIMIT 0, 1000	25 row(s) returned	0.000 sec / 0.000 sec	
✓ 598	09:57:16	SELECT * FROM FACT_REVIEW LIMIT 0, 1000	132 row(s) returned	0.000 sec / 0.000 sec	
✗ 599	10:03:32	SELECT * FROM fact_feedback LIMIT 0, 1000	Error Code: 1146. Table 'duplicatee_commercecompanydb.fact_feedba...	0.000 sec	
✓ 600	10:08:56	CREATE PROCEDURE etl_run_project2() BEGIN -- STEP 0: guarant...	0 row(s) affected	0.016 sec	

2. **Handle unmatched rows.** Project 1 left 12 reviews with no customer. A fact table cannot hold a NULL foreign key to a dimension, so add an "Unknown" member to the customer dimension and point those rows at it. Explain why this is better than dropping them.

*2.Handle unmatched rows. Project 1 left 12 reviews with no customer.

A fact table cannot hold a NULL foreign key to a dimension, so add an "Unknown" member to the customer dimension and Explain why this is better than dropping them.*/

```

ALTER TABLE OrdersFact ADD CONSTRAINT uq_ordersfact_review UNIQUE (ReviewID);
ALTER TABLE FACT_TICKETS ADD CONSTRAINT uq_fact_tickets_ticket UNIQUE (ticket_id);

```

Why this beats dropping the 12 rows: the fact table's job is to hold one row per event (one row per review). if you drop unmatched reviews, your row count silently disagrees with Reviews, your average rating and revenue totals are quietly wrong, and nobody looking at the warehouse later knows those 12 events ever existed. Pointing them at "Unknown Customer" keeps the review itself, its rating, and its revenue in every aggregate – you just can't slice that revenue by customer. It's also reversible: if better customer data shows up later, you re-run the ETL and those rows re-match automatically instead of being gone for good./

3. **Reconcile after loading.** Prove the warehouse agrees with the source by comparing the fact row count against the review count, and total revenue or average rating against the operational figure. A warehouse nobody has reconciled is a warehouse nobody should trust.

```
207
208 /*3.Reconcile after loading. Prove the warehouse agrees with the source by comparin
209 the review count, and total revenue or average rating against the operational figur
210 is a warehouse nobody should trust.*/
211 -- Row counts
212 • SELECT
213     (SELECT COUNT(*) FROM Reviews) AS source_reviews,
214     (SELECT COUNT(*) FROM OrdersFact) AS fact_rows,
215     CASE WHEN (SELECT COUNT(*) FROM Reviews) = (SELECT COUNT(*) FROM OrdersFact)
216         THEN 'PASS' ELSE 'FAIL' END AS result;
217
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
source_reviews	fact_rows	result	
10	10	PASS	

```
18 -- Average rating
19 • SELECT
20     (SELECT ROUND(AVG(Rating),4) FROM Reviews) AS source_avg_rating,
21     (SELECT ROUND(AVG(Rating),4) FROM OrdersFact) AS fact_avg_rating,
22     CASE WHEN ROUND((SELECT AVG(Rating) FROM Reviews),4) = ROUND((SELECT AVG(Rating) FROM OrdersFact),4)
23         THEN 'PASS' ELSE 'FAIL' END AS result;
24
25 -- Revenue proxy, matched rows only (Unknown sentinel has Price = 0.00 so it can't infla
26 • SELECT
27     (SELECT SUM(p.Price) FROM Reviews r JOIN Products p ON r.ProductID = p.ProductID) AS
28     (SELECT SUM(Price) FROM OrdersFact WHERE ProductID <> -1) AS fact_revenue;
29
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
source_avg_rating	fact_avg_rating	result	
4.4000	4.4000	PASS	

```

224
225 -- Revenue proxy, matched rows only
226 • SELECT
227     (SELECT SUM(p.Price) FROM Reviews r JOIN Products p ON r.ProductID = p.ProductID)
228     (SELECT SUM(Price) FROM OrdersFact WHERE ProductID <> -1) AS fact_revenue;
229

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	source_revenue	fact_revenue		
▶	1613.43	1613.43		

```

229
230 -- Ticket count vs staging
231 • SELECT
232     (SELECT COUNT(*) FROM stg_support_ticket) AS staged_tickets,
233     (SELECT COUNT(*) FROM FACT_TICKETS) AS fact_tickets;
234
235

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	staged_tickets	fact_tickets		
▶	25	25		

```

234 SELECT
235     (SELECT COUNT(*) FROM stg_support_ticket) AS staged_tickets,
236     (SELECT COUNT(*) FROM FACT_TICKETS) AS fact_tickets;
237
238 /*Note= If the rows counts and averages match, I proof the warehouse tells the same story as the source
239 , that's the whole point of doing this before trusting a single dashboard number built on top of it*/

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	staged_tickets	fact_tickets		
▶	25	25		

4. **Make it re-runnable.** Running your ETL twice must not double the fact table. Show the row count before and after a second run.

```

237  /*4.Make it re-runnable. Running your ETL twice must not double the fa
238  before and after a second run.*/
239  • SELECT COUNT(*) AS ordersfact_before FROM OrdersFact;
240  • SELECT COUNT(*) AS tickets_before FROM FACT_TICKETS;
241
242  • CALL etl_run_project2();
243
244  • SELECT COUNT(*) AS ordersfact_after FROM OrdersFact;
245  • SELECT COUNT(*) AS tickets_after FROM FACT_TICKETS;
246

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	ordersfact_before			
▶	10			

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	tickets_before			
▶	25			

```

244 • SELECT COUNT(*) AS ordersfact_after FROM OrdersFact;
245 • SELECT COUNT(*) AS tickets_after FROM FACT_TICKETS;
246

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	ordersfact_after			
▶	10			

```

244 • SELECT COUNT(*) AS ordersfact_after FROM OrdersFact;
245 • SELECT COUNT(*) AS tickets_after FROM FACT_TICKETS;
246

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	tickets_after			
▶	25			

5. **Describe your pipeline as ETL or ELT, and say which stage does the transforming and why you put it there.**

```
/*5. Describe your pipeline as ETL or ELT, and say which stage does the transforming and why you put it there.*/  
/*This is ETL, not ELT. The transformaing happens at the staging to final promotion step, cleaning rating, normalizing scores, deriving categories from SKU prefixes, resolving customer/produccct matches, substituting Unknown for NULLs, and all of that happens before a rows lands in OrdersFact or FactTickets. Nothing downstream needs to 'clean up after itself:' what's in the facct and dim tables is already report ready. That's the deliberate choice, the fact tables are what PowerBI/Tableau will query directly, so the transform work belongs upstream of them, not left for the BI layer to redo every time someone opens a dashboard.*/
```

6. **Written answers — one page is enough. Number them to match the task**

```
/*6. Written answers — one page is enough. Number them to match the task:  
1.5 - the API compared to the file sources= and what breaks if it is unavailable.  
The file sources (CSV/JSON/XML) were hard in a different way than the API: getting LOAD DATA LOCAL INFILE and LOAD XML INFILE working meant fighting MySQL's secure_file_priv and local_infile security settings, once I understood why that lockdown exists, it made sense, but finding the setting, disabling it, then remembering to turn it back off afterward cost real time. The API was a different kind of friction: it wasn't a straightforward package install, and I lost time hunting for the mock API files and figuring out how to invoke the script correctly (I initially tried running the Python script directly in the terminal instead of PyCharm, which threw errors until I ran it properly). The actual pagination/JSON-parsing logic was the most technically demanding part once I found the right setup.  
What breaks if the API is unavailable during a load: unlike a CSV file sitting on disk, the API is a live dependency, if it's down or rate-limits me mid-pull, the staging table gets a partial page of tickets with no signal that anything is missing. Since my ETL treats staging as the full source-of-truth snapshot for that run, a partial pull would look like a legitimate (smaller) batch rather than a failure, silently understating ticket counts and reconciliation totals unless I explicitly check the API's reported total record count against what actually landed in staging.*/  
  
/*2.2 - your grain statement, in one sentence = Each row in OrdersFact represents one customer's rating of one product from one review, on one date; each row in FACT_TICKETS represents one support ticket opened by one customer, about one product, on one date.  
  
2.4 - why a warehouse uses surrogate keys = Each row in OrdersFact represents one customer's rating of one product from one review, on one date; each row in FACT_TICKETS represents one support ticket opened by one customer, about one product, on one date.  
  
2.5 - what goes wrong without conformed dimensions= If OrdersFact and FACT_TICKETS each had their own copy of DIM_PRODUCT, the same SKU could get different category labels, different surrogate keys, or diverge over time as one copy gets updated and the other doesn't. Any query trying to compare or join across both facts (example, "products with both low ratings and many tickets") would either fail to join cleanly or silently produce wrong answers, because "product X" in one fact table isn't guaranteed to be the same row as "product X" in the other.  
  
2.6 - the join count for each of your two queries =  
Project 1 normalized query (Reviews = Products): 1 join  
Star schema query (OrdersFact = Dim_Date): 1 join  
Both happen to be 1 join here because OrdersFact denormalizes Category directly onto the fact row, so no separate product dimension join is needed — that's a deliberate simplification, but in a fully normalized star schema this query would typically need a DIM_PRODUCT join too, making the star schema version more joins for this particular question, not fewer. The join-count advantage of a star schema shows up more clearly on questions that touch several dimensions at once (customer, product, and date together), where the normalized schema's foreign-key chains get deeper.
```

3.2 – why an Unknown member beats dropping unmatched rows = A fact table's job is to preserve one row per real-world event. Dropping the 11 unmatched reviews would silently shrink the row count below the source Reviews table, and any aggregate, average rating, revenue, review counts by category, would be quietly wrong with no way to detect it later. Pointing them at an "Unknown Customer" dimension row keeps the event (and its rating/revenue) in every aggregate that doesn't require a customer breakdown, keeps the fact-to-source row count reconcilable, and is reversible: if better customer-matching data shows up later, re-running the ETL re-attaches those rows automatically instead of them being gone for good.

```
3
4 3.5 – ETL or ELT, and where the transforming happens= This pipeline is ETL. The transforming, cleaning ratings, normalizing
5 scores, deriving product categories from SKU prefixes, resolving customer/product matches, substituting Unknown members for NULLs,
6 happens at the staging-to-final promotion step, before anything lands in OrdersFact or FACT_TICKETS. I put it there because those
7 fact/dimension tables are what MYSQL tools query directly; keeping them already clean means every downstream report gets
8 consistent numbers without redoing the same transformation logic later.
9 */
10
```

Deliverables

One archive, [lastname_firstname_project2.zip](#):

1. **API script** — the code that fetches and loads the tickets, with the pagination and retry logic visible.
2. **Star schema diagram** — fact and dimension tables with their relationships. Hand-drawn is acceptable if legible.
3. **SQL scripts** — warehouse tables, the ETL that fills them, your reconciliation checks, and the two side-by-side queries from Part 2.6.
4. **Written answers** — one page is enough. Number them to match the task:
 - o 1.5 – the API compared to the file sources, and what breaks if it is unavailable.
 - o 2.2 – your grain statement, in one sentence.
 - o 2.4 – why a warehouse uses surrogate keys.
 - o 2.5 – what goes wrong without conformed dimensions.
 - o 2.6 – the join count for each of your two queries.
 - o 3.2 – why an Unknown member beats dropping unmatched rows.
 - o 3.5 – ETL or ELT, and where the transforming happens.

Files

- [mock_api.py](#) — the local API server. Needs Python 3.9+ and nothing else. Start it before you run your fetch script.
- [api_pages/](#) — the same responses saved as files. Use these only if you cannot run the server; note it if you did, because pagination and retry still have to be demonstrated.